

Índice.

Índice.	1
Clase Window.	2
Métodos.-	2
Atributos.-	3
Archivo de configuración “.json” por defecto.	3
Clase Processor.	4
Métodos.-	4
Atributos.-	4
Archivo de configuración “.json” por defecto.-	5
Clase Utils.	6
Decorador singleton.	6
Herramientas auxiliares.-	6

Clase Window.

La clase **Window** utiliza una función decorador para permitir una única instancia de la clase, y contiene lo necesario para ejecutar la interfaz de usuario del programa y todos los eventos relacionados. Para ello, se compone de:

Métodos.-

- **__init__ (*arguments)**: Constructor de la clase Window. Se le pueden pasar **argumentos** como **-ui <archivo de configuración de la ui>** y **-counter <archivo de configuración de los contadores>**, desde los que cargará la información de la UI y la configuración para gestionar y procesar correctamente los contadores.
- **__process_args (*arguments)**: Es un **método privado** que comprueba y gestiona los diferentes argumentos que se le haya pasado a la aplicación. Permite los pares de valores mencionados previamente.
- **__set_events_connections ()**: Es un **método privado** que establece las conexiones de los diferentes eventos, entre los botones de la UI y las funciones internas del programa.
- **__load_event ()**: Es un **método privado** que gestiona el **evento** lanzado a la hora **de abrir una imagen** concreta. Se encarga de resetear los valores de la UI para sustituir una anterior ejecución por la nueva imagen. Adapta la imagen al viewer de la UI y carga la imagen en la variable "__image__" de la instancia de Window. Devuelve True si el proceso ha sido exitoso y False en caso contrario. Hace uso de la clase Utils.
- **__reset ()**: Es un **método privado** que gestiona el **evento** lanzado a la hora **de reiniciar** la ejecución. Limpia la UI eliminando todas las imágenes y textos de una ejecución anterior.
- **__clip_event ()**: **Método privado** que gestiona el **evento de recortar** y mostrar a los contadores en la UI. Retorna True si el proceso ha sido exitoso y False en caso contrario. Hace uso de la clase Processor.
- **__extract_event ()**: **Método privado** que gestiona el **evento de extracción** de los números de un contador y mostrarlos en la UI. Retorna True si el proceso de extracción ha sido exitoso y False en caso contrario. Hace uso de la clase Processor.
- **__global_event ()**: **Método privado** que gestiona el **evento de ejecución global**. Permite lanzar los eventos de cargar, recortar y extraer de forma secuencial. Devuelve True si los procesos han sido exitosos y False en caso contrario.
- **__adapt_to_viewer (viewer, frame)**: Es un **método privado** que **permite adaptar un frame o imagen** (de tipo numpy.ndarray) **al tamaño de un viewer** concreto (de tipo QLabel). Retorna la imagen (numpy.ndarray) escalada al tamaño del viewer dado.
- **__set_image_to_viewer (viewer, frame)**: **Método privado** que **permite establecer una imagen en un viewer** concreto. Devuelve True si se ha establecido correctamente, False en caso contrario.

Atributos.-

- **utils**: Objeto de la clase Utils. Permite cargar archivos de configuración "json" y gestionar la lectura de los diccionarios.
- **processor**: Objeto de la clase Processor. Permite gestionar el funcionamiento de la lógica de la aplicación, añadiendo una capa de abstracción y separando la UI de la lógica.
- **ui_config**: Diccionario de la configuración de la interfaz de usuario. Es cargado desde la clase Utils con la ruta "config/ui_config.json" por defecto, pero su ruta puede venir dada como argumento a la hora de ejecutar el programa con el formato "-ui <dirección relativa al archivo de configuración>".
- **ui**: Diseño de Qt cargado. Es la instancia de la interfaz del usuario.
- **__image__**: Almacena la imagen cargada y que está siendo utilizada por la aplicación.
- **DEBUG_MODE**: Valor booleano que permite definir si se quiere mostrar información de depuración o no.

Archivo de configuración ".json" por defecto.

NOTA: La aplicación debería de ser llamada desde la raíz del proyecto, al mismo nivel que el archivo "mainwindow.ui" o que la carpeta "images".

```
1  {
2      "ui": {
3          "title": "Contadores",
4          "ui_relative_path": "mainwindow.ui"
5      },
6      "images": {
7          "folder_relative_path": "images/sources",
8          "template_relative_path": "images/templates",
9          "load_dialog_caption": "Select an image",
10         "load_dialog_filter": "Images (*.png *.xpm *.jpg)"
11     }
12 }
```

Clase Processor.

La clase **Processor** utiliza una función decorador para permitir una única instancia de la clase, y contiene lo necesario para ejecutar la lógica del programa. Para ello, se compone de:

Métodos.-

- **__init__(template_path)**: Es el constructor de la clase Processor. Permite de forma opcional definir una ruta desde la que cargar los templates, ya que por defecto usará "images/templates".
- **set_config(config_path)**: Permite cambiar el archivo de configuración de los contadores. Por defecto usará "config/counters.json".
- **reset()**: Permite resetear la instancia Processor. Devuelve True si se ha restablecido correctamente, False en caso contrario.
- **get_counters_in_frame(frame)**: Obtiene la lista de contadores que se pueden observar en la imagen. Recibe una imagen o frame de tipo "numpy.ndarray" y devuelve una lista. Hace uso del archivo de configuración para tomar los recortes con las medidas y posiciones indicadas.
- **get_counters_saved()**: Permite obtener la última lista de contadores que han sido procesados, es decir, aquellos que han sido recortados.
- **extract_digits(counter)**: Obtiene los dígitos del contador dado por parámetro como "cv2.Mat". Devuelve una cadena de caracteres con el valor final obtenido. Si uno de los dígitos no se pudo detectar correctamente, entonces será sustituido por "?".

Atributos.-

- **REAL_TIME_REACTIVE**: Valor booleano que indica si se puede permitir hacer cambios en el archivo de configuración de los contadores y que sean aplicados en tiempo de ejecución o no.
- **DEBUG_MODE**: Valor booleano que indica si debe mostrar información de depuración o no.
- **__utils__**: Instancia de la clase Utils para cargar archivos de configuración ".json" y gestionar el manejo del diccionario de configuración.
- **__templates__**: Lista de imágenes utilizadas como template. Las imágenes deben de estar compuestas por el valor que formará la salida seguido por barra baja y un valor numérico que los distinga. Así, cuando se utilice una imagen que sea un 2, si la imagen es la quinta imagen que contiene el valor 2, debería de tener el valor "2_5".
- **__counters_config__**: Diccionario con la configuración de los contadores.
- **__counters_frames__**: Lista de imágenes de los contadores que había en un frame.

Archivo de configuración ".json" por defecto.-

```
1  {
2      "digits_per_counter": 4,
3      "decimals_after_coma": 1,
4      "score_threshold": 0.6,
5      "max_overlap": 0.6,
6      "counters": [
7          {
8              "x": 285,
9              "y": 155,
10             "width": 115,
11             "height": 45
12         },
13         {
14             "x": 285,
15             "y": 233,
16             "width": 115,
17             "height": 45
18         },
19         {
20             "x": 283,
21             "y": 315,
22             "width": 116,
23             "height": 45
24         }
25     ]
26 }
```

Clase Utils.

Es una clase que aplica el decorador @singleton y que permite las siguientes funciones:

- **getAbsolutePath(relativePath)**: Permite obtener una ruta absoluta a partir de una ruta relativa desde el directorio padre. En caso de indicar una ruta absoluta, devolverá la misma sin ningún cambio.
- **loadJson(path)**: Permite cargar un archivo json desde una ruta absoluta. Si la ruta es relativa se transforma a absoluta mediante la función anterior. Devuelve el diccionario de datos generado a partir del ".json".
- **getValueOf(dict, key, *keys)**: Permite recorrer un diccionario completo en busca del valor indicado según sus claves. Puede ser un dato de primer nivel o de niveles inferiores, recorriendo todas las claves dadas. En caso de error devolverá el valor None.

Decorador singleton.

Se trata de un decorador para las clases que permite obtener una única instancia de la clase que lo implemente.

Se aplica el decorador de la forma: "@singleton" delante de la clase que quiere implementar el decorador.

Herramientas auxiliares.-

Se ha desarrollado un pequeño programa auxiliar llamado "rename.py" que permite adaptar un dataset de imágenes a la entrada esperada por el programa principal. Se encuentra en la carpeta ".dev".

La herramienta consiste en cambiar el nombre de la imagen al nombre que se espera que tenga la aplicación. Para ello, es necesario pasarle como argumento la ruta de una carpeta con las imágenes a transformar.

La carpeta dada como argumento deberá de estar dividida en subcarpetas que clasifican las imágenes. Las imágenes deberán de estar clasificadas según el contenido de la imagen. Por ejemplo, si tenemos inicialmente una lista de imágenes que contienen los números 0, 1, 2, ..., 8, 9, se deberá de crear una carpeta por cada número, y en la carpeta 0 deberán de estar todas las imágenes que incluyan un 0, y así sucesivamente.

El programa recorrerá dichas subcarpetas y establecerá el nombre apropiado a cada imagen.